EX 2 : MULTI-LAYER PERCEPTRON (MLP) FOR CLASSIFICATION

AIM:

To Develop a Multi-Layer Perceptron (MLP) for a simple classification task. Experiment with different numbers of hidden layers and activation functions, and evaluate the model's performance using accuracy and loss.

OBJECTIVES:

1. Understand the structure and working of a basic feedforward neural network.
2. Experiment with various hidden layer configurations and activation functions Use the sigmoid activation function and cross-entropy loss.
3. Train the MLP using the Iris dataset and evaluate its accuracy and loss.
4. Visualize training performance and use the trained model for prediction.

ALGORITHM:

1. Import the required libraries, including NumPy, Matplotlib, scikit-learn, and TensorFlow/Keras.
2. Load the Iris dataset containing flower features and species labels.
3. Separate the input features and target labels, and encode the target labels using OneHotEncoder.
4. Normalize the input features using StandardScaler to improve model training.
5. Split the dataset into training and testing sets using an 80:20 ratio.
6. Define an MLP model with configurable hidden layers and activation functions such as ReLU, tanh, and sigmoid.
7. Compile the model using the Adam optimizer, categorical cross-entropy loss, and accuracy metric.
8. Train the model using different hidden layer configurations and activation functions for 50 epochs.
9. Evaluate each trained model using test accuracy and test loss.
10. Plot training and validation accuracy and loss curves to visualize model performance.
11. Accept new flower measurements from the user and scale the input using the fitted StandardScaler.

12. Predict the flower species using the trained model and display the predicted class name.

SOURCE CODE:

```
# Step 1: Import Necessary Libraries

import matplotlib.pyplot as plt
import numpy as np

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# ============================================================
# Step 2: Load and Preprocess Data
# ============================================================

# Load Iris dataset
iris = load_iris()

# Input features
X = iris.data

# Target values
y = iris.target.reshape(-1, 1)

# Class names
classes = iris.target_names

# One-hot encode target values
encoder = OneHotEncoder(sparse_output=False)
y_encoded = encoder.fit_transform(y)

# Standardize input features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y_encoded,
    test_size=0.2,
    random_state=42
)

print("Dataset loaded successfully!")
print("Training samples:", X_train.shape[0])
```

```
print("Testing samples:", X_test.shape[0])

# ============================================================
# Step 3: Define MLP Model Creation Function
# ============================================================

def create_mlp(input_dim, output_dim, hidden_layers, activation='relu'):

    model = Sequential()

    # First hidden layer
    model.add(
        Dense(
            hidden_layers[0],
            input_dim=input_dim,
            activation=activation
        )
    )

    # Additional hidden layers
    for units in hidden_layers[1:]:
        model.add(
            Dense(
                units,
                activation=activation
            )
        )

    # Output layer
    model.add(
        Dense(
            output_dim,
            activation='softmax'
        )
    )

    # Compile model
    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

# ============================================================
# Step 4: Train and Evaluate MLP with Various Configurations
# ============================================================

hidden_layer_configs = [
    [8],
    [16, 8],
    [32, 16, 8]
```

```
]

activations = [
    'relu',
    'tanh',
    'sigmoid'
]

# Store results
results = []

# Variables for the final model
last_model = None
last_history = None
last_hidden_layers = None
last_activation = None

# Train models with different configurations
for hidden_layers in hidden_layer_configs:

    for activation in activations:

        print(
            f"\nTesting MLP with "
            f"hidden_layers={hidden_layers}, "
            f"activation={activation}"
        )

        # Create MLP model
        model = create_mlp(
            input_dim=4,
            output_dim=3,
            hidden_layers=hidden_layers,
            activation=activation
        )

        # Train model
        history = model.fit(
            X_train,
            y_train,
            epochs=50,
            batch_size=5,
            verbose=0,
            validation_split=0.1
        )

        # Evaluate model
        test_loss, test_acc = model.evaluate(
            X_test,
            y_test,
            verbose=0
        )
```

```
        print(f"Test Accuracy: {test_acc:.4f}")
        print(f"Test Loss: {test_loss:.4f}")

        # Store results
        results.append(
            (
                hidden_layers,
                activation,
                test_acc,
                test_loss
            )
        )

        # Save last trained model and history
        last_model = model
        last_history = history
        last_hidden_layers = hidden_layers
        last_activation = activation

# ============================================================
# Display Results of All Configurations
# ============================================================

print("\n" + "=" * 60)
print("FINAL RESULTS")
print("=" * 60)

for hidden_layers, activation, accuracy, loss in results:

    print(
        f"Hidden Layers: {hidden_layers}, "
        f"Activation: {activation}, "
        f"Accuracy: {accuracy:.4f}, "
        f"Loss: {loss:.4f}"
    )

# ============================================================
# Step 5: Plot Accuracy and Loss Curves
# ============================================================

plt.figure(figsize=(10, 4))

plt.suptitle(
    f"Config: {last_hidden_layers}, "
    f"Activation: {last_activation}",
    fontsize=14
)

# ---------------------------
# Accuracy Plot
# ---------------------------

plt.subplot(1, 2, 1)
```

```
plt.plot(
    last_history.history['accuracy'],
    label='Train Acc'
)

plt.plot(
    last_history.history['val_accuracy'],
    label='Val Acc'
)

plt.xlabel('Epoch')
plt.ylabel('Accuracy')

plt.title('Model Accuracy')

plt.legend()

# ----------------------------
# Loss Plot
# ----------------------------

plt.subplot(1, 2, 2)

plt.plot(
    last_history.history['loss'],
    label='Train Loss'
)

plt.plot(
    last_history.history['val_loss'],
    label='Val Loss'
)

plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.title('Model Loss')

plt.legend()

plt.tight_layout()

plt.show()

# ============================================================
# Step 6: Predict on New Input
# ============================================================

print("\n" + "=" * 60)
print("IRIS SPECIES PREDICTION")
print("=" * 60)
```

```
# Get input from user
sepal_length = float(
    input("Sepal length (cm): ")
)

sepal_width = float(
    input("Sepal width (cm): ")
)

petal_length = float(
    input("Petal length (cm): ")
)

petal_width = float(
    input("Petal width (cm): ")
)

# Create input array
user_input = np.array([
    [
        sepal_length,
        sepal_width,
        petal_length,
        petal_width
    ]
])

# Scale the input
user_input_scaled = scaler.transform(user_input)

# Predict using the last trained model
prediction = last_model.predict(
    user_input_scaled,
    verbose=0
)

# Find class with highest probability
predicted_class_index = np.argmax(prediction)

# Get class name
predicted_class_name = classes[
    predicted_class_index
]

# Display prediction
print(
    f"\n Predicted Iris Species: "
    f"{predicted_class_name}"
)

print(
    f"Prediction Probabilities: "
    f"{prediction[0]}")
```

OUTPUT:

```
Dataset loaded successfully!
Training samples: 120
Testing samples: 30

Testing MLP with hidden_layers=[8], activation=relu
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Test Accuracy: 0.9333
Test Loss: 0.1967

Testing MLP with hidden_layers=[8], activation=tanh
Test Accuracy: 1.0000
Test Loss: 0.1790

Testing MLP with hidden_layers=[8], activation=sigmoid
Test Accuracy: 0.9333
Test Loss: 0.3845

Testing MLP with hidden_layers=[16, 8], activation=relu
Test Accuracy: 1.0000
Test Loss: 0.1080

Testing MLP with hidden_layers=[16, 8], activation=tanh
Test Accuracy: 1.0000
Test Loss: 0.0899

Testing MLP with hidden_layers=[16, 8], activation=sigmoid
Test Accuracy: 0.9333
Test Loss: 0.4520

Testing MLP with hidden_layers=[32, 16, 8], activation=relu
Test Accuracy: 1.0000
Test Loss: 0.0298
```

```
Testing MLP with hidden_layers=[16, 8], activation=sigmoid
Test Accuracy: 0.9333
Test Loss: 0.4520

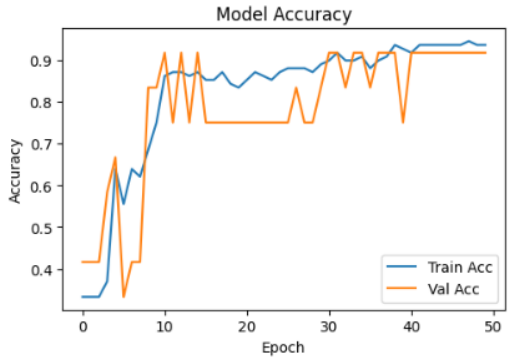
Testing MLP with hidden_layers=[32, 16, 8], activation=relu
Test Accuracy: 1.0000
Test Loss: 0.0298

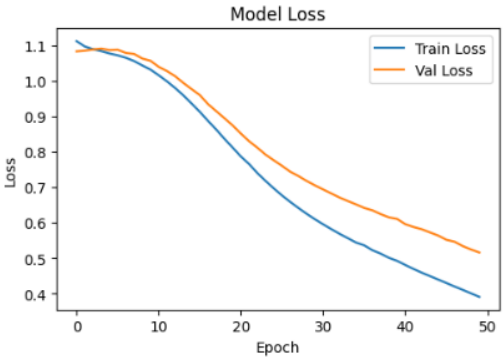
Testing MLP with hidden_layers=[32, 16, 8], activation=tanh
Test Accuracy: 1.0000
Test Loss: 0.0541

Testing MLP with hidden_layers=[32, 16, 8], activation=sigmoid
Test Accuracy: 0.9333
Test Loss: 0.3813

============================================================
FINAL RESULTS
============================================================
Hidden Layers: [8], Activation: relu, Accuracy: 0.9333, Loss: 0.1967
Hidden Layers: [8], Activation: tanh, Accuracy: 1.0000, Loss: 0.1790
Hidden Layers: [8], Activation: sigmoid, Accuracy: 0.9333, Loss: 0.3845
Hidden Layers: [16, 8], Activation: relu, Accuracy: 1.0000, Loss: 0.1080
Hidden Layers: [16, 8], Activation: tanh, Accuracy: 1.0000, Loss: 0.0899
Hidden Layers: [16, 8], Activation: sigmoid, Accuracy: 0.9333, Loss: 0.4520
Hidden Layers: [32, 16, 8], Activation: relu, Accuracy: 1.0000, Loss: 0.0298
Hidden Layers: [32, 16, 8], Activation: tanh, Accuracy: 1.0000, Loss: 0.0541
Hidden Layers: [32, 16, 8], Activation: sigmoid, Accuracy: 0.9333, Loss: 0.3813
```

Config: [32, 16, 8], Activation: sigmoid






```
============================================================
IRIS SPECIES PREDICTION
============================================================
```

RESULT:

This experiment successfully demonstrates the use of MLPs for a multi-class classification problem using the Iris dataset. Through experimentation with different hidden layer depths and activation functions, students gain a practical understanding of neural network design and optimization techniques.